01

DevOps Agent

Jenkins 빌드 실패와 배포 후 파드 이상의 원인을 Slack 스레드에 답하는 봇

역할	설계, 개발, 권한 설계, 운영 (단독)
환경	Python, Jenkins, EKS, ArgoCD, Slack
결과	빌드 실패 23종, 배포 후 파드 이상 8종은 LLM 없이 즉시 답변 클러스터 권한은 읽기 전용 QA, 운영 두 환경에서 배포 후 파드 확인 동작

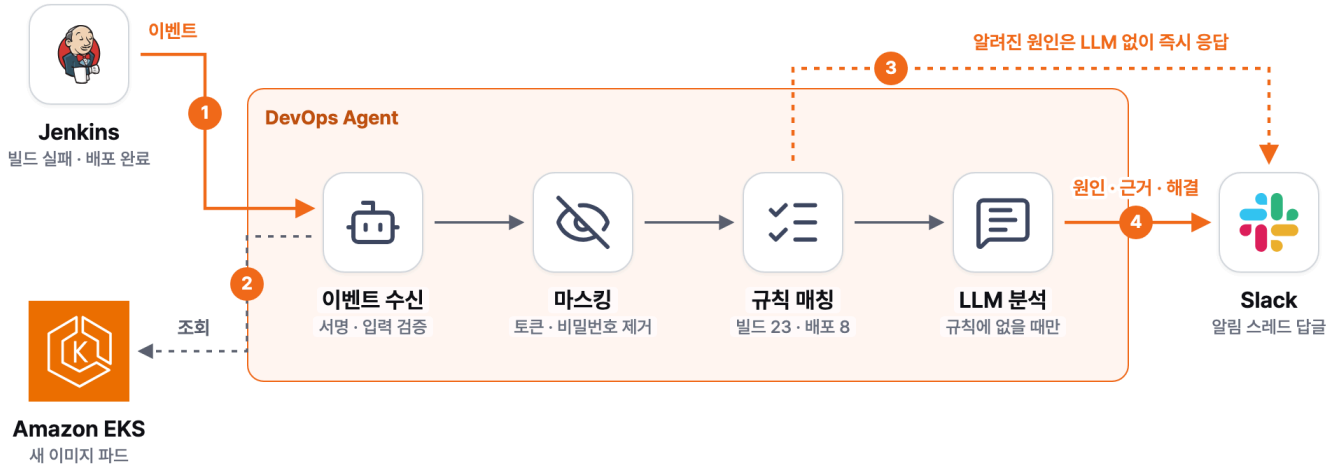
문제와 구조

문제

- 빌드가 실패할 때마다 콘솔 로그를 열어 원인 확인
- 배포 후 새 파드가 CrashLoop여도 옛 파드가 서비스를 유지해 인지 어려움

동작

- 기존 알림은 유지하고 스레드 답글만 봇이 작성
- 빌드 주소로 해당 알림을 찾아 동시 실패에도 답글이 섞이지 않음
- 알려진 원인은 규칙으로 답하고 나머지만 LLM에 전달
- 배포 직후 새 이미지 파드를 2초 간격으로 확인, 이상 시 즉시 분석



1 이벤트 수신
Jenkins가 서명한 이벤트를 보내면 서명과 입력을 검증

2 읽기 전용 조회
콘솔 로그와 새 이미지 파드의 상태만 조회. 쓰기 권한 없음

3 규칙으로 즉시 응답
마스킹 후 알려진 원인은 LLM 없이 바로 답글

4 LLM 분석
규칙에 없을 때만 원인, 근거, 해결 방법을 정리해 답글

AI에 맡기는 범위와 사람이 확인하는 지점

맡긴 일

- 실패 로그를 읽고 원인 추정
- 원인의 근거가 된 로그 줄과 해결 순서 제안
- 서버 접속이나 명령 실행은 불가. 로그를 읽고 답하는 것까지만 담당
- AI가 근거라고 든 로그 줄이 실제 로그에 없으면 확신도를 낮게 표시
- 답글 맨 아래에 "AI 추정이라 틀릴 수 있음" 표기

사람이 확인하는 일

- 재시작이나 설정 변경 같은 해결 조치는 사람이 실행
- 봇이 클러스터에서 할 수 있는 일은 조회뿐. Secret 읽기와 명령 실행은 불가
- 시크릿 반영과 재시작은 별도 권한으로 분리. 해당 네임스페이스에만 적용되고, 채널 멤버만 실행하며 결과는 채널에 공개
- 신규 서비스 설정은 봇이 초안만 생성하고 검토 후 커밋은 사람이 진행
- Secret 읽기가 실제로 거부되는지 점검 명령으로 직접 확인

02

DB Migration

운영 Aurora MySQL 5.7을 8.0으로 업그레이드하고, QA 데이터 동기화를 덤프 배치에서 실시간 복제로 전환

역할	업그레이드 주도, QA 동기화 구조 설계와 구축 (성능 개선 일부는 개발팀과 협업)
환경	Aurora MySQL, DMS Serverless, QA, 운영
결과	QA, 운영 모두 8.0.42 업그레이드 RDS 월 비용 57% 절감, 연장 지원 비용 100% 제거 QA 동기화 1일 1회 덤프에서 변경 즉시 복제로 전환 (테이블 268개)

업그레이드 전 준비

5.7 표준 지원 종료로 연장 지원 요금 발생 (월 DB 비용의 약 53%). 문제 발생 시 원인이 버전인지 부하인지 가리기 어려워 성능 안정화를 먼저 진행.

성능 안정화

작업	결과
읽기 트래픽 Reader 분산	**Writer 집중 부하 감소**
커넥션 풀을 실사용량 기준으로 축소	**전체 커넥션 1,450 → 303**
3초 이상 쿼리 가시화, 튜닝	**QA Read DB CPU 피크 100% → 24%**
운영 Writer 덤프 배치 중단	**CDC로 대체**

Reader 분산과 커넥션 풀 축소는 개발팀과 협업. 쿼리 튜닝은 가시화와 검증 담당.

사전 검증

- 스키마 정적 분석 (MySQL Shell `util.checkForServerUpgrade()`): Fatal 0건, Warning 194건
- 사용 중인 테이블 전수 조사: 16% 삭제, 72% 수정
- 8.0 비호환 정리: 예약어 컬럼명, utf8mb3 → utf8mb4, Zero Date 기본값, 동작이 바뀐 비밀번호 함수 쿼리
- 환경별 파라미터 그룹 대조. 놓치면 `max_connections` 등이 기본값으로 복귀
- 서버 문자셋과 collation 명시

운영 업그레이드와 QA 동기화

운영 업그레이드 절차

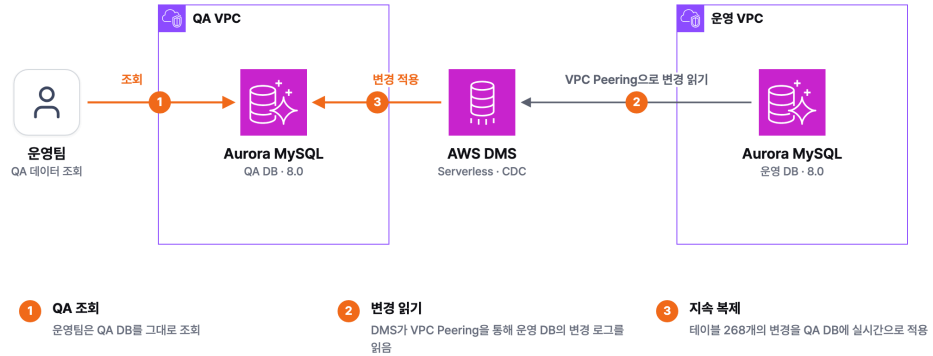
사전 공지 → 점검 화면 → 스냅샷 → 충돌 제거 → 1차 확인 → 8.0.42 → 2차 확인 → 재개

- 스냅샷은 스크립트 실행 전과 버전 업그레이드 전에 각각 확보
- 점검 화면은 ALB 고정 응답(앱 JSON, 웹 HTML 503)으로 준비해 QA에서 리허설
- 프리체크 실패 시 로그에서 에러 항목만 추출하는 절차 정리. 실패해도 5.7로 남아 다운타임 없음
- 롤백은 스냅샷 복원뿐이며 엔드포인트가 바뀐다는 점을 사전 공유

업그레이드 후 발견한 장애

증상	원인과 조치
알림 API 200, 발송 없음	구버전 JDBC 드라이버가 8.0에서 제거된 변수를 조회. 호환 드라이버로 교체
자동 배정 API 500 반복	8.0의 정규식 실행 시간 제한 초과. 파라미터 조정, 쿼리 개선은 후속 등록

QA 동기화: 덤프 배치에서 DMS CDC로



- 운영과 QA의 VPC 대역이 같아 Peering 불가. QA DB를 다른 대역으로 옮겨 Peering 후 인터넷을 거치지 않고 복제
- DMS가 테이블, 컬럼 코멘트를 복제하지 않는 제약 확인, 재동기화 절차 정리
- 업그레이드 중 CDC 지연이 최대 21분까지 늘었다가 자동 회복, 데이터 손실 없음

03

GitOps 구축

서버에 SSH로 산출물을 보내던 배포를 EKS 위 ArgoCD 기반 GitOps 배포로 전환하고, 서비스마다 달랐던 Jenkins 파이프라인을 공용 라이브러리로 통일

역할	플랫폼 설계와 구축, 파이프라인 표준화
환경	EKS, Terraform, Jenkins, Nexus, ArgoCD, Helm, External Secrets
결과	신규 서비스 배포에 추가하는 설정 파일 2개 Shared Library 원자 함수 7개, 조합 템플릿 4개 Terraform 커스텀 모듈 15종 프론트엔드 이미지 1.09GB에서 292MB로 축소 (Multi-stage)

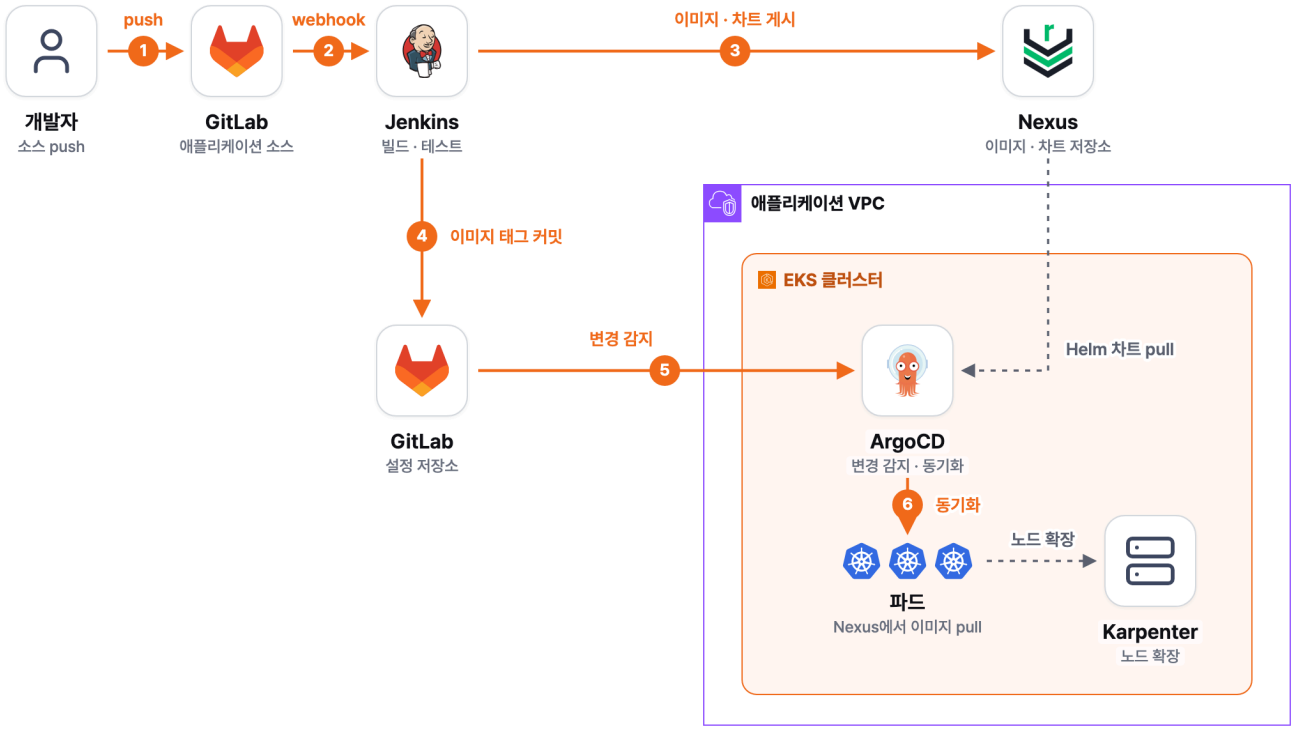
배경과 배포 흐름

기존 방식

- Jenkins가 SSH로 서버에 산출물을 보내는 Push 방식
- 서비스마다 파이프라인이 달라 어떤 버전이 어디에 있는지 Git으로 추적 불가
- 설정 파일에 DB 비밀번호와 AWS 키가 커밋된 서비스 존재

전환 기준

- Jenkins는 빌드와 Git 커밋까지만, 클러스터 반영은 ArgoCD 전담
- Git이 배포 기록이므로 롤백은 커밋 revert
- AWS는 Terraform, 쿠버네티스 안은 ArgoCD로 경계 분리



1 소스 push
개발자가 애플리케이션 소스를 GitLab에 push

2 CI 트리거
GitLab webhook이 Jenkins를 실행하고, 공용 라이브러리 파이프라인으로 빌드·테스트

3 산출물 게시
Docker 이미지와 공통 라이브러리, Helm 차트를 Nexus에 게시

4 태그 커밋
Jenkins가 설정 저장소의 이미지 태그를 갱신하고 커밋

5 변경 감지
EKS 안의 ArgoCD가 설정 저장소의 변경을 감지하고, Nexus에서 차트를 pull

6 클러스터 동기화
매니페스트를 적용하면 Karpenter가 필요한 노드를 늘리고, 파드가 이미지를 pull

설계 선택

App-of-Apps

- root Application이 apps 디렉터리를 재귀 등록. 신규 서비스는 `Application.yaml`과 `values.yaml` 추가로 끝
- ApplicationSet 대신 개별 Application 선택. 서비스마다 네임스페이스, 차트 버전, values 구조가 다름
- root를 앱용(prune 사용)과 플랫폼용(prune 미사용)으로 분리. AppProject로 개발자와 CI 계정은 앱 범위만 접근

Jenkins Shared Library

- 원자 함수 7개: checkout, 이미지 빌드, push, 이미지 정리, GitOps values 갱신, ArgoCD sync, 알림. 함수끼리 호출하지 않음
- 조합 템플릿 4개: QA 자동 배포, 운영 빌드, 운영 배포(수동 승인), 수동 검증
- 운영은 Build와 Deploy Job 분리. Deploy가 실행 시점의 Nexus 최신 태그를 조회하고 승인 게이트 통과 후 진행

시크릿과 설정값 분리

- 시크릿은 Secrets Manager와 External Secrets, 설정값은 GitOps values의 ConfigMap
- AWS 자격 증명은 키 없이 EKS Pod Identity. IRSA와 달리 공용 라이브러리 수정 없이 적용
- 설정 파일의 평문 기본값을 6개 서비스에서 제거, 정적 액세스 키 문자열 0건 확인

Terraform과 ArgoCD 관리 경계

쿠버네티스 API 밖이면 Terraform, 안이면 ArgoCD. 예외는 부트스트랩(ArgoCD 자신, Karpenter). Traefik, External Secrets, ALB Controller, Gateway API CRD를 Terraform에서 ArgoCD로 이관.

04

모니터링 체계 구축

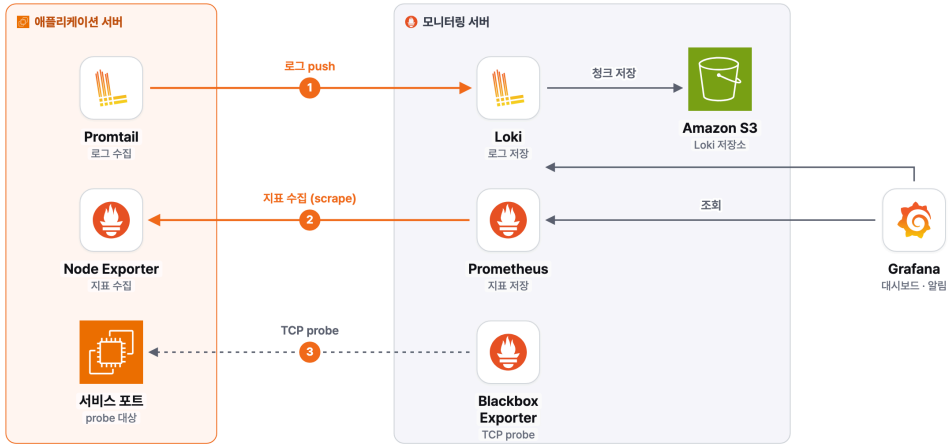
자체 제작 사내 관제를 Prometheus, Grafana, Loki로 교체하고 운영 서버 30대에 적용

역할	대안 검증, 설계, 운영 서버 30대 적용, 알림 체계 설계
환경	Prometheus, Grafana, Loki, AWS SSM
결과	운영 서버 30대 일괄 적용 서비스 DOWN을 앱 포트 40여 개 단위로 직접 확인 서버당 수집 에이전트 메모리 약 80MB, 지표 수집 주기 15초(기존 60초 폴링)

구성과 수집 방식

기존 관제의 한계

- 서버 1대의 자체 제작 시스템, 디스크 사용률 92%
- 서비스 DOWN은 "수집 실패" 알람으로만 간접 확인
- 시험한 상용 APM이 서버 8대에서 OOM Kill을 일으켜 철수



- 모니터링 서버는 공인 IP 없는 사설 서브넷, Docker Compose 4개 컨테이너에 CPU, 메모리 상한 지정
- Loki 저장소는 S3, 접근은 키 없이 인스턴스 프로파일. 수집 포트는 모니터링 서버 보안 그룹만 허용

수집 에이전트 비교 (실측)

구성	메모리	판단
Node Exporter + Promtail	약 80MB	**채택**
Grafana Alloy (통합)	235~255MB	제외

운영 서버 다수가 메모리 1GB급이고 대부분 Swap이 없어 소형 인스턴스 부담이 적은 쪽을 선택. 관리 대상 2개로 늘지만 장애 위험이 낮음. 모니터링 스택도 실측해 필요 메모리를 약 5GB로 산정.

서비스 DOWN 감지

- 운영 서버 30대의 앱 포트 40여 개를 Blackbox Exporter로 직접 TCP 확인. `probe_success == 0` 이면 DOWN
- 알림에 서버와 서비스 라벨을 붙여 어느 서버의 어떤 앱인지 즉시 식별
- 대상 목록은 리스닝 포트, 프로세스 작업 디렉터리, 프로세스 관리 도구 상태, 인프라 문서로 교차 확인

알림과 배포 자동화

기존 알림 기준 이전

레거시 관제의 이벤트 13종, 임계치, 채널, 발송 시간대를 조사해 Grafana 규칙과 1:1 매핑표 작성 후 전환. 임계치는 3단계에서 2단계로 축소.

분류	조건	수준
리소스	CPU, 메모리, 디스크 각 90% 초과	critical
에러 로그	5분간 에러 로그 1건 이상	warning
서비스 DOWN	앱 포트 TCP 확인 실패	critical
RDS	CPU 70% / 90% 초과	warning / critical

알림 피로 관리

- 복구 알림 필요 여부로 Contact Point를 나눠 type 라벨 4종으로 라우팅
- 에러 로그 알림은 10분 반복 억제, 복구 알림 없음
- 서버 이름, 현재 값, 해당 서버 대시보드 링크 포함

배포와 자동화

- AWS SSM Run Command로 운영 30대에 일괄 설치. 설치, 서비스 등록, 설정, 활성화, 확인 순서 고정
- Promtail 시작 전에 오프셋 파일을 미리 기록해 과거 로그 전체 재수집 방지
- 서버별 로그 대시보드를 Terraform으로 생성. Loki API의 서버 목록을 `for_each`로 받고, UID 고정과 `overwrite = false`로 기존 대시보드 보호
- 새 알림은 테스트 채널에서 검증 후 운영 채널로 이동. 테스트 로그 주입부터 수신까지 확인

이전과 이후

항목	이전	이후
서비스 DOWN	수집 실패로 간접 확인	**앱 포트 단위 직접 확인**
에러 감지	앱이 직접 전송	**Loki 쿼리 기반**
대시보드 추가	화면에서 수동 복제	**Terraform 자동 생성**